pfi

Python for Fixed Income

COURSE 1 · PYTHON FOUNDATIONS

NumPy, Part 2

WEEK 2, DAY 1, PART 2

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

By the end of this part you can

Select by a condition

Below par, high yield, premium or discount.

```
tickers[prices < 100]
```

Hold a table in one array

Rows first, then columns.

```
bond_table[:, 1]
```

Price a bond from its cash flows

Under simple assumptions, stated plainly.

```
(cash_flows * discount_factors).sum()
```

Save and load arrays

And check that they arrived intact.

```
np.array_equal(loaded_table, bond_table)
```

A mask selects by condition, not by position



Count and total (COUNTIF, SUMIF)

```
below_par.sum()
2
faces[below_par].sum()
2250000
```

Combine conditions with & and |

```
tickers[(scores >= 11)
        & (prices > 102)]
['DNMR']
```

Label every bond (IF, filled down)

```
np.where(prices > 100,
  'Premium', 'Discount')
```

A slice of an array is a window, not a copy

The change to the slice lands on prices

```
first_two = prices[0:2]
first_two[0] = 0
print(prices)
[  0.   103.25  95.   101.  ]
```

The change to a copy stays in the copy

```
first_two_copy = prices[0:2].copy()
first_two_copy[0] = 0
print(prices)
[ 98.5  103.25  95.   101.  ]
```

A slice of a list is a new list. A slice of an array shares its numbers with the original, and nothing warns you.

To change a slice and keep the original as it is: slice, then .copy().

A table in one array: rows first, then columns

bond_table has shape (4, 3): 4 rows and 3 columns

axis=0

row	0: coupon	1: price	2: face
0 QVNE	5.25	98.5	1500000
1 DNMR	6.125	103.25	500000
2 PLWK	4.75	95.0	750000
3 HLVF	7.5	101.0	250000

axis=1

Two indexes: row, then column

```
print(bond_table[1, 1])
103.25
print(bond_table[:, 1])
[ 98.5  103.25  95.   101.  ]
```

axis=0

Down the rows. One result per column.

In Excel, a SUM at the bottom of each column.

axis=1

Across the columns. One result per row.

In Excel, a SUM at the end of each row.

Cash flows times discount factors, summed

	period 1	period 2	period 3	...	period 14
`cash_flows`	2.625	2.625	2.625	...	102.625
`* discount_factors`	0.9732	0.9471	0.9217	...	0.6835
`= present_values`	2.5546	2.4861	2.4195	...	70.1474

`present_values.sum()`
SUMPRODUCT in Excel

→ **98.5067**

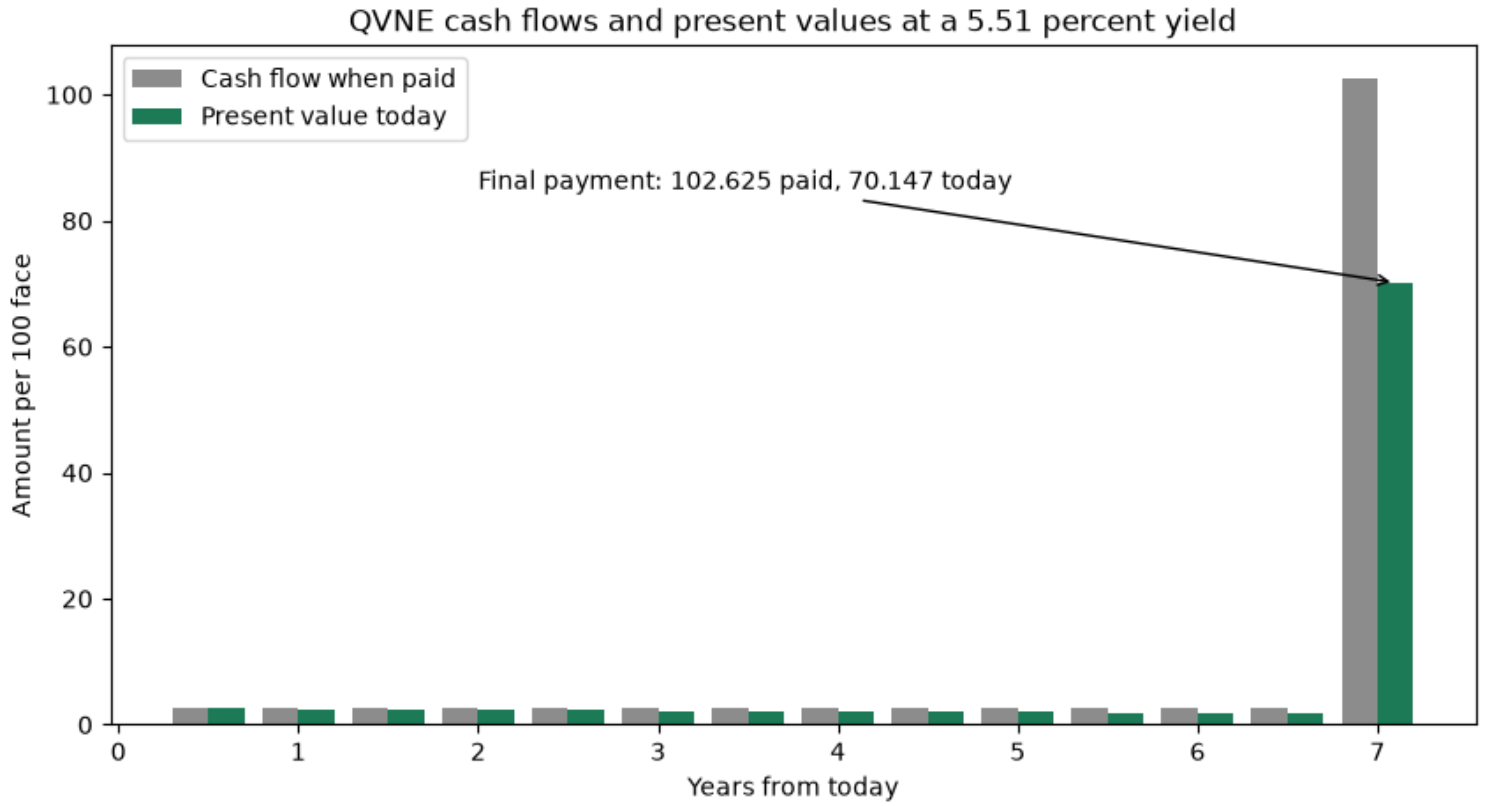
Three assumptions, stated plainly

Whole periods. Semiannual compounding. Settlement on a coupon date.

The full treatment belongs to the Bond Math course.

```
present_values = cash_flows * discount_factors
qvne_price = present_values.sum()
Price of QVNE at a yield of 5.51 percent: 98.5067
```

Cash flows against present values



Grey bars

The cash flows as paid: 13 coupons of 2.625, then 102.625 at 7 years

Green bars

The same payments worth less today. They sum to the price, 98.5067.

The last bar

The principal is 70.147 of the 98.507 price

Read the picture and check it against the numbers in the notebook.

Save it, load it, check it

prices

Four prices, in memory

`np.save()` →

desk_prices.npy

Exact. Excel cannot open it.

`np.load()` →

loaded_prices

Back exactly as saved

bond_table

Coupon, price, and face

`np.savetxt()` →

desk_bond_table.csv

Plain text. Opens in Excel.

`np.loadtxt()` →

loaded_table

Read back, header skipped

```
coupon,price,face
5.250,98.500,1500000.000
6.125,103.250,500000.000
4.750,95.000,750000.000
7.500,101.000,250000.000
```

```
print(np.array_equal(loaded_table, bond_table))
True
```

No tickers in the file. Text beside numbers is a job for pandas.

The AI check

Every notebook ends with code written in the style of an AI assistant. It has one bug. Find it.

1

Read

Read the code before you run it.

2

Predict

Decide what it should produce.

3

Run

Run it and look at the output.

4

Check

Compare with a number you worked out yourself.

Code that runs is not the same as code that is right.

What you can do now

- Pick out bonds by a condition and label them with np.where()
- Hold a table in one array, and reshape, turn, and value it with @
- Price a bond from its cash flows and discount factors
- Save arrays, load them back, and check that they arrived intact

Tomorrow pandas: Series and DataFrames, and the portfolio grows to two hundred bonds